2026-11-05 React 프로젝트의 Tailwind CSS 4와 Vite

- 날짜: 2026-11-05
- 앱: `lgcns-fe`
- 패키지: `vite` ^8.3.5, `react` ^19.3.1, `tailwindcss` ^4.3.4, `@tailwindcss/vite` ^4.3.4 (`lgcns-fe/package.json`)
- 기준: 이 문서의 "현재 코드"는 지금 디스크의 파일이다. 저장소에 없는 설정 파일은 있는 것처럼 쓰지 않는다.

이 프로젝트에는 `tailwind.config.js`가 없다. Tailwind 4를 Vite 플러그인으로 붙였고, 진입 CSS는 `@import "tailwindcss"` 한 줄이다. 화면의 간격, 색, 글자 크기는 컴포넌트의 `className` 문자열이다.

1. 파일이 언제 실행되는가

소스 `lgcns-fe/index.html`에는 스타일시트 `<link>`가 없다. 스크립트는 하나다.

```
<script type="module" src="/src/main.tsx"></script>
```

`async`도 `defer`도 적혀 있지 않다. `type="module"`인 스크립트는 HTML 명세에서 기본적으로 지연 실행이다. 받는 동안 문서 파싱을 막지 않고, 문서 파싱이 끝난 뒤 `DOMContentLoaded` 전에 실행된다. 같은 문서의 모듈 스크립트는 적힌 순서를 지킨다. 이 파일에는 모듈 스크립트가 하나다.

파일	선언	실행 시점	HTML script와의 차이
lgcns-fe/index.html	<script type="module" src="/src/main.tsx">. async, defer 속성 없음	브라우저가 HTML을 파싱한 뒤 이 모듈을 실행한다	고전 스크립트(type 없음)는 파서를 막는다. async는 받는 즉시 실행하고 순서가 없다. defer는 파싱 뒤 순서대로 실행한다. type="module"은 그 defer와 같이 늦춰지고, import/export를 쓴다
lgcns-fe/package.json	"type": "module"	Node가 이 패키지의 .js를 ES 모듈로 다룬다. Vite가 vite.config.ts를 읽을 때 이 설정이 적용된다	브라우저 스크립트 태그가 아니다. index.html의 type="module"과는 다른 단계다
lgcns-fe/vite.config.ts	export default defineConfig({ plugins: [react(), tailwindcss()] })	npm run dev 또는 npm run build가 Vite를 띄울 때, 요청을 받기 전에 한 번 평가된다	페이지 HTML에 붙지 않는다. 브라우저가 나중에 받는 CSS를 만드는 도구 설정이다
lgcns-fe/src/main.tsx	import './index.css' 다음 createRoot(...).render(...). export 없음	모듈 그래프의 시작점이다. CSS import가 평가된 뒤 React가 #root에 그린다	개발 중에는 Vite가 이 모듈을 브라우저에 넘긴다. 파일 첫머리에 "use client"는 없다. 이 앱은 Next가 아니다
lgcns-fe/src/index.css	@import "tailwindcss";	Vite와 @tailwindcss/vite가 이 파일을 CSS로 바꿀 때 처리한다. 브라우저가 Tailwind 컴파일러를 실행하지 않는다	<link rel="stylesheet">를 소스 HTML에 직접 적지 않았다. 번들이 스타일을 붙인다
lgcns-fe/src/lib/ui.ts	const 문자열 다음 export { authCard, authMain, errorText, fieldLabel, outlineButton, primaryButton, textInput }	Login.tsx, Signup.tsx, ClientLogin.tsx, SiteHeader.tsx가 import할 때 평가된다. 값은 className에 들어가는 문자열이다	default export가 없다. CSS 파일이 아니다
lgcns-fe/src/App.tsx	export default function App()	main.tsx가 import한 뒤 렌더 중에 호출된다	App.css를 import하지 않는다. 그 파일은 디스크에 있어도 이 모듈 그래프에 없다

npm run build가 만든 lgcns-fe/dist/index.html은 소스와 다르다. 그 빌드 결과에는 모듈 스크립트와 스타일 링크가 둘 다 있다.

```
<script type="module" crossorigin src="/assets/index-cFAIldT1.js"></script>
<link rel="stylesheet" crossorigin href="/assets/index-Ca0DnIZk.css">
```

파일 이름의 해시는 그 빌드의 출력이다. 다시 빌드하면 바뀐다. 개발 서버의 소스 HTML에는 이 `<link>`가 없고, 프로덕션 HTML에는 있다. 어느 쪽이든 Tailwind 변환은 브라우저 밖, Vite 플러그인 안에서 끝난다.

참고: HTML script 요소 (https://html.spec.whatwg.org/multipage/scripting.html#the-script-element), MDN script (https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements/script), MDN link rel=stylesheet (https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Attributes/rel#stylesheet), Vite CSS (https://vite.dev/guide/features.html#css), React createRoot (https://react.dev/reference/react-dom/client/createRoot).

2. 플러그인과 @import

`vite.config.ts`는 `tailwindcss`를 `@tailwindcss/vite`에서 default import 한다. `react`는 `@vitejs/plugin-react`의 default import다. `defineConfig`의 `plugins` 배열 순서는 `react()` 다음 `tailwindcss()`다. JSX 변환과 CSS 변환은 서로 다른 파일을 다룬다. `tailwindcss`와 `@tailwindcss/vite`는 `devDependencies`에 있다. 버전은 둘 다 ^4.3.4다. 설치된 `@tailwindcss/vite`의 peer는 `vite` ^5.2.0, ^6, ^7, ^8이다. 이 앱의 Vite는 ^8.3.5다.

`src/index.css` 전체는 다음 한 줄이다.

```
@import "tailwindcss";
```

이 문장은 패키지 `tailwindcss`의 `index.css`를 가져온다. 그 파일의 첫 층은 이렇다.

```
@layer theme, base, components, utilities;
```

그 다음 `@layer theme` 안에 `@theme default`가 색, 간격, 글꼴 변수를 둔다. `@layer base`는 Preflight다. `*`, `::before`, `::after`에 `box-sizing: border-box`, `margin: 0`, `padding: 0`, `border: 0 solid`를 준다. `@layer utilities` 안의 `@tailwind utilities` 자리가, 소스에서 발견된 클래스만 유틸리티 규칙으로 바뀌는 자리다.

CSS 캐스케이드 레이어는 먼저 선언한 레이어가 나중 레이어보다 약하다. 여기 순서는 `theme`, `base`, `components`, `utilities`다. 같은 우선순위에서 유틸리티가 base보다 이긴다. 이 앱은 `@theme`으로 토큰을 더 올리거나, `components` 레이어에 컴포넌트 클래스를 쓰지 않는다.

참고: Tailwind Vite 설치 (https://tailwindcss.com/docs/installation/using-vite), MDN @import (https://developer.mozilla.org/en-US/docs/Web/CSS/@import), MDN @layer (https://developer.mozilla.org/en-US/docs/Web/CSS/@layer), MDN box-sizing (https://developer.mozilla.org/en-US/docs/Web/CSS/box-sizing).

3. 클래스 문자열을 소스에서 찾는다

Tailwind는 브라우저에서 클래스 이름을 만들지 않는다. 빌드 때 소스 텍스트에 적힌 유틸리티 이름을 찾는다. 이 앱의 이름은 두 곳에 있다.

1. JSX `className="..."` 안의 완성된 이름. 예: `SiteHeader`의 `className="mx-auto flex w-full max-w-7xl items-center justify-between gap-10 px-4 py-8"`.
2. `src/lib/ui.ts`의 문자열 리터럴. `Login`, `Signup`, `ClientLogin`, `SiteHeader`가 그 상수를 `className`에 넣는다.

`ui.ts`는 named export만 있다. `primaryButton` 같은 값 안에는 `h-14`, `rounded-xl`, `bg-[#2563eb]`, `hover:bg-[#1d4ed8]`, `disabled:opacity-60`처럼 유틸리티 이름이 통째로 적혀 있다. `authCard`에는 `min-[480px]:px-12`가 있다. 이것은 최소 너비 480px부터 `px-12`를 적용하는 variant다. 임의의 값 `h-[52px]`, `text-[#374151]`, `max-w-[484px]`, `shadow-[0px_4px_6px_rgba(37,99,235,0.2)]`도 소스에 그대로 있다.

헤더 로그아웃 버튼은 두 문자열을 붙인다.

```
className={`${outlineButton} h-10 px-3 text-sm`}
```

`outlineButton`이 가리키는 이름들은 `ui.ts` 리터럴에 있고, `h-10`, `px-3`, `text-sm`은 이 JSX 소스에 있다. 스캐너는 런타임에 이어 붙인 결과를 실행해 보지 않고, 각 파일에 적힌 토큰을 본다. `hover:` 뒤에 변수만 두고 색 이름을 실행 중에 조립하는 코드는 이 프로젝트에 없다.

`SiteHeader`의 링크 클래스는 파일 안 상수다.

```
const linkClass = "text-[#111827] aria-[current=page]:font-semibold";
```

`NavLink`는 현재 주소일 때 `aria-current="page"`를 둔다. 그 속성값일 때만 `font-semibold`가 적용된다. 모바일 메뉴 버튼은 `md:hidden`, 데스크톱 목록은 `hidden ... md:flex`다. `md`는 Tailwind 기본 중단점이다.

`className="sr-only"`는 스크린리더 전용으로 남기는 유틸리티다. 버튼의 보이는 아이콘은 `aria-hidden="true"`인 `svg`다.

참고: Tailwind가 소스에서 클래스를 찾는 방법 (https://tailwindcss.com/docs/detecting-classes-in-source-files), React Router NavLink (https://reactrouter.com/api/components/NavLink), MDN aria-current (https://developer.mozilla.org/en-US/docs/Web/Accessibility/ARIA/Reference/Attributes/aria-current).

4. 이 화면이 실제로 쓰는 클래스

`App`은 주소를 나눈다. 스타일 진입과는 별개로, `className`을 가진 화면은 아래 파일이다.

파일	ui.ts에서 가져오는 이름	그 파일에 직접 적은 예
src/pages/Login.tsx	authMain, authCard, fieldLabel, textInput, primaryButton, errorText	구글 버튼 h-14 w-full rounded-xl border ..., 카카오 버튼 bg-[#fee500]
src/pages/Signup.tsx	위 이름과 outlineButton	단계 글자 flex-1 text-center text-xs font-bold text-[#2563eb]
src/pages/ClientLogin.tsx	fieldLabel, textInput, primaryButton	폼 mx-auto flex w-full max-w-sm flex-col gap-4 p-8
src/components/SiteHeader.tsx	outlineButton	헤더 mx-auto flex w-full max-w-7xl ...
src/pages/Home.tsx	없음	배너 제목 text-center text-2xl font-bold text-red-500, 버튼 rounded-lg bg-[#171717] ... text-white
src/pages/About.tsx, Contact.tsx	없음	섹션 flex flex-col gap-2 px-4 py-8 text-left
src/pages/Feed.tsx, Blog.tsx, Product.tsx, Cart.tsx, Order.tsx	없음	바깥 p-4 text-left. 피드의 Next 버튼은 w-fit rounded-lg border border-[#d1d5db] px-4 py-2

로그인 카드의 바깥은 authMain이다. flex min-h-dvh items-center justify-center bg-[#f3f4f6] px-4 py-10이다. min-h-dvh는 CSS min-height: 100dvh에 대응한다. dvh는 동적 뷰포트 높이다.

Preflight가 h1의 기본 크기와 여백을 지운 뒤, 로그인 제목은 text-[28px] font-bold text-[#111827]로 다시 정한다. App.css의 #root { width: 1126px } 같은 규칙은 App.tsx가 그 파일을 import하지 않으므로 이 그래프에 실리지 않는다.

참고: MDN min-height (https://developer.mozilla.org/en-US/docs/Web/CSS/min-height), MDN Viewport-percentage lengths (https://developer.mozilla.org/en-US/docs/Web/CSS/length#viewport-percentage_lengths), Tailwind Preflight (https://tailwindcss.com/docs/preflight).